Resync-and-Anchor Copy Drafting for Lossless Code Fast-Apply

Novain Shibu (independent) · October 2026 · Technical report · Code and data: `resync-apply` (Apache-2.0) · DOI: *pending*

Abstract

AI code editors apply an edit by having a model rewrite the whole file. Most of that output is copied from the original, so speculative decoding can propose long copied spans and let the model verify them in one forward pass, with output equal to greedy decoding. We study how few verification passes a copy drafter can get away with. Our drafter verifies up to eight copy branches per pass as a token tree. The candidates come from four sources: *resync* positions after a mismatch (the model replaced, inserted or skipped one source token), *1-token anchors* ranked by distance, read frontiers, and branch lengths taken from a learned table.

We tested it on 199 real edits from the Kortix FastApply dataset with the open Kortix/FastApply-1.5B model. It needs **~1.27x fewer verification passes than tuned SuffixDecoding**, the strongest open method and the one built into vLLM. On the 169 edits never used for tuning the ratio is **1.26–1.28x (bootstrap 95% CI 1.19–1.36)**. On a free NVIDIA T4 in a Hugging Face Python loop this becomes only **1.05x lower wall-clock** (56.8 s vs 59.9 s on 30 edits). Over the first 5 edits it is about **37x faster than normal decoding**. Output matched greedy decoding on every edit compared (Section 5.4). We also report seven ideas that did not work, a loss breakdown against a copy oracle, and a laptop simulator that reproduces GPU pass counts exactly.

1. Introduction

"Fast apply" is the step where an AI code editor turns a short edit snippet into a complete updated file. Cursor popularised doing this with a dedicated model plus "speculative edits": the original file serves as the draft. Commercial services (Morph, Relace) and OpenAI's Predicted Outputs offer similar features. Open-source servers rely on model-free drafters: prompt lookup decoding (n-gram matches into the prompt) or SuffixDecoding (Snowflake Arctic Inference; a suffix tree over the prompt and earlier outputs, with adaptive draft length), which ships in vLLM.

When the model's output stops matching the current copy position, a copy drafter must find where to copy from next. Every wrong guess costs one full forward pass. This report measures that cost and shows that a small set of cheap, rule-based candidates removes about a fifth of all passes compared with tuned SuffixDecoding. The contributions:

1. A multi-branch copy drafter with resync candidates and 1-token anchors, evaluated against tuned SuffixDecoding on a real fast-apply model and dataset, including a held-out split.
2. A laptop simulator that reproduces GPU pass counts exactly and wall-clock within ~1–2 s. Each idea is scored in seconds.
3. Negative results for seven ideas, and a breakdown of the remaining gap to a copy oracle.

2. Related work

- **Copy and retrieval drafting.** LLMA / Inference with Reference (2023) copies the reference span with the longest matching prefix. Prompt lookup decoding (2023) does n-gram matching into the prompt. REST (retrieval trie + tree verification). SAM Decoding (suffix automaton, longest match). Graft (2026) fills pruned draft-tree slots with retrieved candidates. SuffixDecoding (Snowflake) uses suffix trees with adaptive speculation length and an optional tree mode.
- **Code editing.** Cursor speculative edits (closed). EfficientEdit (ASE 2025): reuses the original code as a linear draft, re-anchors by matching the tail of the generated text to a prefix of the remaining original code, and uses an edit-oriented draft model for new text (10–13x vs autoregressive decoding; it does not compare against SuffixDecoding). QiMeng-PRepair (2026). An empirical study of speculative decoding on software-engineering tasks (2026) finds SuffixDecoding strongest on editing and repair.
- **Reuse of rejected predictions and trees.** Token Recycling, Lookahead/Jacobi decoding, SpecInfer/Medusa tree verification, FlashInfer cascade attention.

What we believe is not in prior work. We read the papers above and EfficientEdit's released code. None of them tries explicit resync positions after a one-token substitution, insertion or deletion as parallel branches. None combines them with 1-token anchors and learned branch sizing, or measures this against SuffixDecoding on fast apply. To our knowledge this is new as a combination. The individual parts (copying, n-gram anchors, tree verification) are known.

3. Pass physics on a T4

Each verification pass is one forward pass of the target model over the drafted tokens. We measured its cost for FastApply-1.5B (fp16, T4) with a 2,048-token context:

drafted tokens	1	8	32	64	128	256	512	1024	2048
ms per pass	38.7	35.7	37.1	37.0	39.2	59.6	108.6	215.2	463.1

The cost is flat up to ~127 drafted tokens, then rises by ~0.2 ms per token. The Hugging Face Python loop adds ~22 ms per pass. Our tree-mask construction adds ~16 ms on tree passes. For our best design on 30 edits the time splits as prompt prefill 20%, fixed per-pass cost 69%, extra drafted tokens 11%. **Passes are the currency:** removing a pass saves ~60 ms, while drafting 100 extra tokens costs almost nothing below the knee.

4. Method

The drafter (`sim/drafter.py`) is one Python class. The CPU simulator and the GPU loop both run it.

1. **Index.** The prompt (original file + edit snippet) is indexed by n-grams for n = 32, 16, 12, 8, 6, 4, 3, 2, 1.
2. **Main branch.** Continue copying at the current pointer. The draft length doubles after a fully accepted pass and shrinks to `2·accepted + 16` after a miss (bounded by kmin/kmax).
3. **Resync candidates (after a mismatch).** If the model emitted token *t* where the source had *s[p]*, propose copying from *p+1* (substitution), *p* (insertion: the model added *t*) and *p+2* (deletion). If *s[p+1] = t*, the deletion branch is tried first.
4. **Anchors.** Positions after the longest n-gram matches of the latest output, longest n first, down to **1-token anchors**. Ties are ranked nearest-after the previous pointer.
5. **Read frontiers.** The furthest position copied so far in the original code and in the snippet, and the next line start after each.

6. **Tree verification.** Up to K = 8 branches are verified together as a token tree (4D attention mask). The KV cache keeps only the accepted path. Acceptance is greedy: a drafted token is kept only if it equals the model's argmax.
7. **Learned branch sizing.** An expected-copy-length table over candidate features (kind, n-gram strength, distance bucket, region, line start), trained on records 100–198. It cuts side-branch lengths (1.0 × expected) and caps the main branch (6 × expected).
8. **Hybrid passes.** A plain single-branch pass when the previous pass was fully accepted, a tree only after a miss.
9. **Fallback.** With no anchor, copy what followed the latest earlier match in the model's own output.

5. Experimental setup

- **Model:** Kortix/FastApply-1.5B-v1.0 (fine-tuned Qwen2.5-Coder-1.5B), fp16, batch size 1, greedy, Hugging Face transformers. **Data:** Kortix/FastApply-dataset-v1.0 test split, 199 edits. Prompts average 1,471 tokens (max 4,679). Outputs total 229,173 tokens. Model and data are both Apache-2.0.
- **Baseline:** SuffixDecoding from arctic-inference 0.3.0 in the same loop, tuned in its favour: max_spec_tokens 256, max_spec_factor 4.0, min_token_prob 0.1, and one cache shared across requests, so it also learns from earlier edits' outputs (our drafter does not). Tree mode (`use_tree_spec`) was also measured.
- **Simulator.** Greedy verification is deterministic given the true output. Pass counts for any drafter can therefore be computed on a CPU from the recorded outputs (`traces/oracle.json`). Simulated SuffixDecoding passes equal the real GPU count (868 on the first 30 edits). Our tree drafter gives 664 simulated = 664 on GPU. Wall-clock is predicted from the measured cost curve plus the 22 ms loop and 16 ms tree costs. The tree run was predicted at 69.5 s and measured at 70.5 s.
- **Splits.** The first 30 edits have GPU wall-clock for every method and were used for tuning. Edits 30–198 were never used for tuning. The sizing table was trained on 100–198, so the strictly clean held-out subset for the learned variant is 30–99. The held-out numbers below use the untuned and the time-tuned *rule* configurations, which use no table.
- **Hardware:** one free Kaggle NVIDIA T4 (16 GB).

6. Results

6.1 Verification passes (simulator, exact)

Configuration	Edits	Ours	SuffixDecoding linear	SuffixDecoding tree
untuned (K=8, strong 64, weak 16)	first 30 (tuning set)	664	868 (1.31x)	843 (1.27x)
untuned	**held-out 30–198**	**3,314**	4,170 (**1.26x**)	4,105 (1.24x)
time-tuned (K=8, s32, w8, kmin 64, kmax 384)	**held-out 30–198**	**3,253**	4,170 (**1.28x**)	4,105 (1.26x)
both	all 199	3,915–3,978	5,038 (1.27–1.29x)	4,948 (1.24–1.26x)

Bootstrap 95% confidence intervals for the held-out ratio vs SuffixDecoding linear: **1.19–1.33** (untuned) and **1.21–1.36** (time-tuned). On the first 30 edits the per-edit result was 14 wins, 11 losses and 5 ties (CI

1.14–1.47). The gain is concentrated in a few large files.

6.2 Ablation (passes; first 30 / all 199)

Drafter	first 30	all 199
single pointer, n-gram re-anchor	979	5,708
+ resync candidates	852	–
+ K=4 short side branches	794	–
+ 1-token anchors, K=8	689	4,175
+ read frontiers	680	4,138
+ strength-weighted branch lengths	**664**	**3,978**
+ learned sizing + hybrid (GPU run)	644	–
SuffixDecoding linear / tree (reference)	868 / 843	5,038 / 4,948

Resync gives the largest single step (979 → 852, −13%). 1-token anchors give the second largest (794 → 689).

6.3 Wall-clock (T4, 30 edits, Hugging Face loop)

Method	Passes	Time	Identical	vs SuffixDecoding
Tree v1 (Python O(q²) mask)	664	160.2 s	30/30	0.42x
Tree, fast numpy mask, time-tuned	662	70.5 s	30/30	0.88x
Learned sizing + hybrid	**644**	**56.8 s**	**30/30**	**1.05x**
Learned sizing + hybrid + cascade attention	644	61.3 s	30/30	0.98x
SuffixDecoding (tuned)	869	59.9 s	30/30	1.00x

Normal greedy decoding of the first 5 edits (6,914 tokens) took 342.8 s. On the same 5 edits our method took 9.2 s (**37x**) and SuffixDecoding 9.9 s (35x). An earlier run on the first 10 edits measured 20.8x for our older single-branch drafter and 23.3x for SuffixDecoding. On a T4 in this loop, the 1.35x pass advantage becomes a 1.05x time advantage, because tree passes cost more (Section 3).

6.4 Output equality

All methods use greedy verification and so should reproduce greedy decoding. Two checks were made. The output was compared with plain `generate` on 10 edits (oracle run, 10/10 equal) and on 5 edits (race run, 5/5 equal). On edits 6–30 of the race all three methods produced identical outputs. We therefore say "matched greedy on every edit compared", not "bit-identical". Rare fp16 near-ties can flip an argmax for any method that changes the batch shape. We saw this on 1–2 of 48 items in earlier runs, for prompt lookup and SuffixDecoding as well.

7. What did not work

Idea	Result
Merge-drafter (deterministically merge original + snippet, verify the predicted file)	1.04x fewer passes than SuffixDecoding. The merge was exact for only 7 of 200 files
Multi-candidate n-gram matching (K up to 16)	1.09x vs SuffixDecoding tree
Structural jump candidates (start of next new snippet line)	1.09x, no gain over multi-candidate

Jacobi residue recycling (reuse predictions after the mismatch)	residue branch won 18 of 654 passes. Divergences are mostly insertions, not substitutions
Learned ranker for candidate order	~1%
Junction forks from a line-level diff	~1% (644 → 638)
Cascade attention (exact log-sum-exp merge)	exact on 4/4 test trees but slower on T4 (61.3 s vs 56.8 s)
Small helper draft model for new text (projection)	new-text passes emit 0.9% of tokens; ceiling 1.02–1.12x fewer passes before paying for the model

8. Remaining headroom

Copy oracle. An oracle that always copies the longest matching span (it knows the future output) needs 374 passes on the first 30 edits (2,118 on all 199), against our 638–664. We classified our passes:

- 249 were as good as the oracle;
- 161 were genuinely new text;
- 191 had the right position among the branches, but the branch was too short;
- 37 lacked the right candidate.

Making all branches long only reaches 592–600 passes, and on a T4 the extra tokens raise time from 65 s to 73–104 s. The rest of the oracle's advantage comes from knowing which of many matches is right, which no cheap rule found.

Other hardware (projection, NOT measured). We replaced the measured curve with first-principles cost models. The predicted ratio vs SuffixDecoding:

- T4 with no loop overhead and a fused tree pass: ~1.10x
- 7B model on 2×T4: ~0.89x (we lose: the flat region shrinks and our extra tree tokens cost)
- H100 with a vLLM-like loop: ~1.22x (1.5B) and ~1.27x (7B)

These numbers only show which way the result should move. Measuring them is future work.

9. Limitations

- One model (1.5B), one dataset, one GPU type, batch size 1, greedy decoding only.
- The tree verifies ~6.6x more tokens per pass than SuffixDecoding. Under heavy batching the GPU is no longer flat in drafted tokens, and the gain may shrink or reverse.
- Wall-clock was measured in a Hugging Face Python loop, not in vLLM or SGLang.
- Settings were tuned on the first 30 edits. Held-out results are given, but the GPU wall-clock run uses those same 30 edits.
- Output equality is shown on the edits compared (Section 6.4), not proven.

10. Reproducibility

- `python reproduce.py` (laptop, ~1 minute, no GPU) prints 3,978 / 664 passes for the drafter and 2,118 / 374 for the copy oracle. `python sim/hybrid.py` predicts T4 wall-clock from the measured curve.
- `traces/oracle.json`: 199 tokenized prompts, the model's outputs, the original GPU timings and the cost curve.

- `kaggle/`: the scripts behind every GPU number (`oracle_run.py`, `gpu_race.py`, `heldout_suffix.py`). About 10–12 GPU hours of free Kaggle quota were used in total.

Acknowledgements and AI-assistance statement

The code, experiments and text were produced with Claude (Anthropic), an AI assistant, under the author's direction. The author chose the problem, set the kill rules and approved every result. Every number in this report comes from a logged run in the repository. Thanks to Kortix for releasing the FastApply model and dataset, and to Snowflake for open-sourcing Arctic Inference.

References

1. Yang et al. *Inference with Reference: Lossless Acceleration of Large Language Models* (LLMA). arXiv:2304.04487, 2023.
2. Saxena. *Prompt Lookup Decoding*. GitHub, 2023.
3. Oliaro et al. *SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications*. arXiv:2411.04975; Snowflake Arctic Inference.
4. *EfficientEdit: Accelerating Code Editing via Edit-Oriented Speculative Decoding*. ASE 2025, arXiv:2506.02780.
5. Hu et al. *SAM Decoding: Speculative Decoding via Suffix Automaton*. ACL 2025, arXiv:2411.10666.
6. *Draft Less, Retrieve More: Hybrid Tree Construction for Speculative Decoding* (Graft). arXiv:2605.20104, 2026.
7. He et al. *REST: Retrieval-Based Speculative Decoding*. NAACL 2024.
8. Miao et al. *SpecInfer*. ASPLOS 2024. Cai et al. *Medusa*. ICML 2024.
9. Fu et al. *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. ICML 2024.
10. *An Empirical Study of Speculative Decoding on Software Engineering Tasks*. arXiv:2604.26469, 2026.
11. *QiMeng-PRepair: Precise Code Repair via Edit-Aware Reward Optimization*. arXiv:2604.05963, 2026.
12. Cursor. *Editing Files at 1000 Tokens per Second* (blog), 2024.